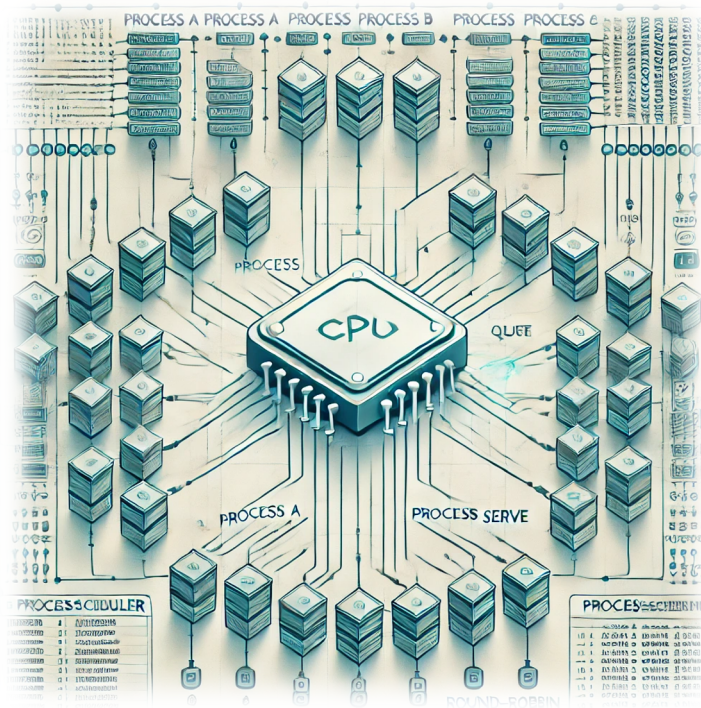


THREADS

The Scheduler



Scheduler Project Definition

(Spring 2024)

1. Introduction

This initial phase of our OS development project builds the kernel's core functionality: the Scheduler. The Scheduler project builds the functionality in the kernel that controls processes at the lowest level. This is the part of the kernel that creates processes, manages which process runs and for how long, manages process state and inter-process relationships, and much more.

1.1. Objectives and Scope

The objective of this document is to specify requirements for the Scheduler development project. Included is a specification for the required functions (e.g. the Scheduler Application Programming Interface (API)) that must be implemented for this phase of the kernel's development.

This document combined with the course's lecture and presentation materials define the totality of the Scheduler project.

1.2. Target Audience

The document is intended for students taking advanced operating systems courses at the graduate or undergraduate level.

2. Scheduler Overview

The primary responsibility of the Scheduler is process management and the initialization of the kernel's core elements. The Scheduler encapsulates the central features needed throughout kernel's lifecycle from boot to process management to system shutdown. This includes:

- Starting or booting the kernel
- Scheduling processes to run
- Managing process states (starting, running, and terminating)
- Timer interrupt handling
- System shutdown

The Scheduler is the first step in an iterative development of the whole kernel where each iteration of the development builds upon the previous. The Scheduler serves as the foundation for the rest of the kernel's major features, see Figure 1.

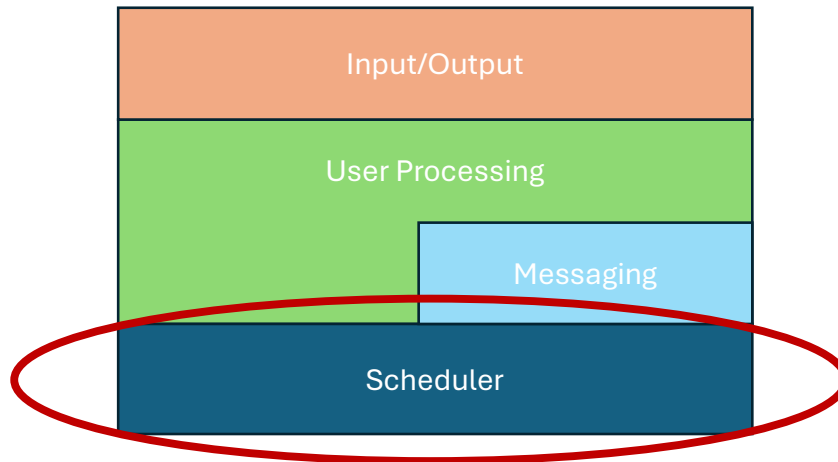


Figure 1 The Scheduler is the foundation of the kernel project.

This interdependency requires each iteration in the project to have a well-defined interface that future layers depend on. For example, we develop the **k_spawn** function in the Scheduler and the **k_spawn** function is used by future functional layers that need to create a *kernel-level* process.



Important: All processing done in the Scheduler is performed in **kernel** mode. This mode must be enforced to prevent the invocation of a Scheduler function in **user** mode.

2.1. Kernel Startup: **bootstrap()**

The OS boots, or starts, by way of a call to the **bootstrap()** function where the kernel is initialized and the system's first processes are created. The **bootstrap()** function is invoked by the THREADS library and is where you begin working on your kernel implementation.

Students are provided a THREADS-Scheduler *Starter Project* that begins with a partially implemented **bootstrap()** function. The Starter Project lets students immediately begin working through the kernel's initialization. As you develop your kernel, start by considering what types of activities need to be performed before running the first process. Much of the success of the Scheduler phase is understanding all its moving parts and the management components (e.g. data and data structures) needed for effective process management.

Consider the following items as a guide for what must be done to boot the kernel:

- Initialize the process table
- Setup interrupt handlers
- Create the watchdog process
- Create the kernel's entry point process
- Start running

The above list is not a complete list of what needs to happen during the boot process but should give you a good idea of where to start.



Important: The `bootstrap()` function does not run in the context of a process within the kernel. It acts as the bootloader, and processing should never return to this function once the first process is started.

The goal at the end of `bootstrap` is to have two operational processes in the kernel and complete the boot operation by setting the processing context to the `SchedulerEntryPoint` process.

3. Kernel Overview

The end of the boot function marks the beginning of kernel operation, as the `SchedulerEntryPoint` process is started. From this point onward, the kernel ensures that a process is always running throughout the system's lifetime. The `SchedulerEntryPoint` is a high priority process that becomes the entry point for the Messaging layer of the kernel. With the Scheduler, this process is used to exercise and test the various Scheduler features.

3.1. Process Management

One of the most important features of the Scheduler is process management and deciding which process should be running at any given time. Process management in the kernel is built around a preemptive, priority-based, round-robin scheduling algorithm, which balances responsiveness and fairness amongst the active processes.

Preemptive, Priority-Based Round Robin Scheduling

The kernel uses a scheduling algorithm that prioritizes processes based on their assigned priority levels. Higher-priority processes are always selected for execution first, ensuring that time-sensitive tasks receive immediate attention. In cases where multiple processes share the same priority level, the Scheduler enforces fairness through a round-robin mechanism. The kernel uses a fixed period, the quantum, to execute a process before the Scheduler switches to the next process. This is time slicing and is where the kernel must decide which process to run when the current process's run time exceeds the quantum.

Timer interrupts play a key role in time slicing management. The kernel receives a signal from the timer at regular intervals through the firing of the timer interrupt. Each timer interrupt triggers the Scheduler to assess whether the currently running process's time slice has expired. If so, the Scheduler initiates a context switch to the next process in the queue *if* there is another process that has the right to run. This decision is based on the kernel's scheduling algorithm.

The Role of the Dispatcher

It is the role of the `dispatcher()` function to decide which process must run next and to make the context switch to the selected process. The dispatcher operates by consulting

the ready queue, which should be organized by priority, to determine the highest-priority process ready for execution. Once the next process to run is identified, the kernel calls the THREADS' `context_switch` function, passing the context of the next process to run. The THREADS library context management features handle the low-level details of saving and restoring process states, so students need only to focus on providing the appropriate context information.



Important: The `dispatcher()` may or may not decide to change from the current context to another. A call to the dispatcher may result in the continuation of the current process.

Interrupt Management and Resource Protection

Interrupts are asynchronous and may occur at any point during the kernel's lifecycle. Therefore, to maintain system stability during process management, the kernel must carefully manage the enabling and disabling of interrupts. Interrupts must be temporarily disabled during critical operations, such as modifying the ready queue or updating the process table. These shared resources are central to the kernel's scheduling functionality and must be protected from race conditions or corruption caused by simultaneous access.



Important: It's important to have interrupts enabled when processing is returned to an active process.

Maintaining Continuity with the Watchdog Process

The `watchdog` process runs at the lowest priority and should only be switched to when there are no other ready processes in the kernel. The watchdog is an idle process that ensures the system remains operational and capable of responding to new events. Although it performs no active computation, the watchdog serves as a placeholder, guaranteeing that the kernel always has a process to execute.

3.2. System Termination and Parent-Child Relationships

All processes, except for the `watchdog` and the `SchedulerEntryPoint`, have a parent process. These two foundational processes are started during the bootstrap phase and operate outside the traditional parent-child hierarchy. Outside of calls to `k_spawn` in the bootstrap function, every process that calls `k_spawn` is the parent of the newly created process.



Important: Proper system shutdown must consider the fact that there are processes without a parent.

Parent-Child Relationships

When a process is created using `k_spawn`, the parent process assumes responsibility for managing its child processes. This relationship ensures a hierarchical structure where processes depend on their parent for lifecycle management. A parent process should not terminate until all its child processes have exited. The kernel enforces this hierarchy by requiring parent processes to wait for their children to finish, using `k_wait` (which

retrieves the exit code of next child process to quit or terminate), before itself can terminate.

It is the responsibility of the parent to ensure proper cleanup and resource deallocation for its child processes before concluding its own execution. This structure prevents orphaned children in the kernel.

Role of the Watchdog in System Termination

The **watchdog** plays a crucial role in ensuring that the system terminates in an orderly manner. As the one and only idle process, the **watchdog** is designed to be scheduled only when no other processes are ready or running. During system termination, it is the responsibility of the **watchdog** to determine whether any other processes remain in the system and then act accordingly. Consider the following two conditions the **watchdog** process will encounter when it runs:

1. **No Remaining Processes:**

This marks the end of the kernel's lifecycle since there are no other active processes in the kernel. In this condition, the **watchdog** concludes the kernel's lifecycle by cleaning up and exiting gracefully.

2. **Remaining Processes:**

Ideally, if the **watchdog** detects that processes remain in the process table, it must decide why it is running as the idle process. In a kernel with input and/or output occurring, it's the role of the idle process to keep the kernel running by waiting for an I/O operation to finish. In the Scheduler, there are no I/O features so there should always be at least one running process or process that is ready to run. Otherwise, this condition is an error condition and the kernel must be halted with a call to the **THREADS stop()** function with an error code indicating there was a problem in the kernel.

The kernel should always have at least one running process up until time of termination. In the case there the watchdog detects active processes, and none are running or ready to run, the kernel is halted as though a deadlock has occurred. This condition represents a flaw in the kernel's logic.

Future iterations of the kernel's termination logic incorporate handling for I/O as well as for blocked processes that may be in a true deadlock state.



Important: *There is a hook that is placed in the Scheduler to extend its functionality for a time when I/O is functional. The **check_io()** function must be implemented and must always return 0. This function is provided for you in the Starter Project.*

4. Scheduler Interface

The Scheduler defines a set of core API functions that future iterations of the OS, with more functionality, can do things like create processes or wait for processes to complete. These must be implemented in your Scheduler project. The Start Project contains prototypes for all of these functions and a partial implementation for some of them.

k_spawn

int k_spawn(char* name, int(*entry_point)(void*), void* arg, int stack_size, int priority);														
Description	The k_spawn function creates a new process in the kernel at the specified <i>priority</i> . The process begins at the function pointed to by <i>entry_point</i> , which is called with a single parameter with the value in <i>arg</i> . The THREADS library manages the stack for each context, but the caller needs to provide the stack size in <i>stack_size</i> when creating the context.													
Arguments	<i>name</i>	The name of the new process, see NOTES for size constraints.												
	<i>entry_point</i>	A function pointer that serves as the entry point for the new process.												
	<i>arg</i>	This must point to a string that is passed to the new process as a parameter to the <i>entry_point</i> function, see the function NOTES.												
	<i>stack_size</i>	The size of the stack used for the process.												
	<i>priority</i>	The new process's priority. The priority values range from 0-5 with 5 representing the highest priority process and 0 being the lowest priority.												
Return Value	int	Upon success, the function returns the process id (pid) of the newly created process. The function should an error code in an error occurs: <table><tr><th>Return Value</th><th>Result</th></tr><tr><td><pid></td><td>Process ID of the new process.</td></tr><tr><td>-1</td><td>A parameter is missing or invalid.</td></tr><tr><td>-2</td><td>The specified stack size is out of range.</td></tr><tr><td>-3</td><td>The specified priority is out of range.</td></tr><tr><td>-4</td><td>The process cannot be created because the process table is full.</td></tr></table>	Return Value	Result	<pid>	Process ID of the new process.	-1	A parameter is missing or invalid.	-2	The specified stack size is out of range.	-3	The specified priority is out of range.	-4	The process cannot be created because the process table is full.
Return Value	Result													
<pid>	Process ID of the new process.													
-1	A parameter is missing or invalid.													
-2	The specified stack size is out of range.													
-3	The specified priority is out of range.													
-4	The process cannot be created because the process table is full.													

NOTES:

The strings pointed to by the *name* and *arg* parameters cannot be larger than **THREADS_MAX_NAME** in size. If either of these strings are larger than **THREADS_MAX_NAME**, the kernel must be halted with a call to **stop** and an error code of 1:

stop(1);

This is the function where the context of the new process is initialized with **context_initialize**. Save the new context in the process table to use when it's time to start the process or return processing to the process via the **context_switch** function.



Important: Interrupts must be enabled when a process is started.

k_wait

```
int k_wait(int* p_child_exit_code);
```

Description	The k_wait function waits for a child process to exit and returns it's exit code. The function returns when one of the caller's child processes quits. The pid of the quitting process is returned by the function. For a process with more than one child, this function should be called once for each child process before itself calls k_exit .										
Arguments	<i>p_child_exit_code</i>	A pointer to an integer value that receives the exit code of the process.									
Return Value	<i>int</i>	The function returns the pid of the quitting child upon successful creation. Otherwise, the return value must be one of the following error codes: <table><tr><th>Return Value</th><th>Result</th></tr><tr><td><pid></td><td>Process ID of the process that exited.</td></tr><tr><td>-1</td><td>There are no child processes available to wait for.</td></tr><tr><td>-5</td><td>The process was signaled while waiting.</td></tr></table>		Return Value	Result	<pid>	Process ID of the process that exited.	-1	There are no child processes available to wait for.	-5	The process was signaled while waiting.
Return Value	Result										
<pid>	Process ID of the process that exited.										
-1	There are no child processes available to wait for.										
-5	The process was signaled while waiting.										

NOTES:

The caller does not specify a child process to wait for. The function will return after any child of the caller terminates. If one or more child processes has already exited when this function is called, then the function should return right away without waiting and with the information (pid and exit code) from one already terminated child.

A parent process must call this function once for every child that was spawned to ensure the proper termination of a process and to clean up any system resources that the child process is or was using.

k_join

```
int k_join(int pid, int* p_child_exit_code);
```

Description	The k_join function waits for the specified process to terminate and retrieves its exit code.								
Arguments	<i>pid</i>	The process ID of the child process to wait for.							
	<i>p_child_exit_code</i>	A pointer to an integer value that receives the exit code of the process.							
Return Value	int	Returns 0 upon successful completion or an error code if the process cannot be joined. <table><tr><th>Return Value</th><th>Result</th></tr><tr><td>0</td><td>Success</td></tr><tr><td>-5</td><td>The process was signaled while waiting.</td></tr></table>		Return Value	Result	0	Success	-5	The process was signaled while waiting.
Return Value	Result								
0	Success								
-5	The process was signaled while waiting.								

NOTES:

Processes cannot join with themselves and cannot join with their parent process. If the process is attempting to join itself or attempting to join a non-existing process, the kernel should be halted with an error code of 1.

stop(1);

If the process attempts to join its parent, the kernel should be halted with an error code of 2.

k_kill

int k_kill(int pid, int signal);		
Description	The k_kill function sends a signal to a specified process, allowing it to handle the signal appropriately. The only signal supported is the SIG_TERM signal defined in THREADS. Once a process is signaled, future calls by the process to the signaled function should return a 1.	
Arguments	<i>pid</i>	The process ID of the process to signal.
	<i>signal</i>	The signal to send to the process. The possible signals are: SIG_TERM – Signal to terminate
Return Value	int	The function returns 0 upon success.

NOTES:

If the specified signal value is not **SIG_TERM** or if there is an attempt to signal a non-existing process, then the kernel should be halted with an error code of 1.

stop(1);

Processes in the Scheduler do not go out of their way to terminate when signaled, this function simply marks the process as signaled, see the function **signaled**. Note the documentation in functions where a process is blocked and see the required return codes if a process is signaled during its wait time. Future iterations of the kernel use this signal to terminate user-level processes.

k_exit

void k_exit(int exit_code);		
Description	The k_exit function terminates the calling process and sets its exit code.	
Arguments	<i>exit_code</i>	The process's exit code to set for the exiting child. This value should be overwritten to -5 when if the child process is in a signaled state.
Return Value	None	The function should never return.

NOTES:

Processes cannot exit if they have one or more active child processes. If a process attempts to quit with an active process, then kernel should be stopped with a code of 1:

stop(1);

It is important to coordinate the termination of a child process with its parent process. *Note that the **SchedulerEntryPoint** and the watchdog processes do not have a parent.*

k_getpid

int k_getpid(void);

Description	The k_getpid function returns the process ID of the calling process	
Arguments	None	
Return Value	int	The ID of the process.

time_slice

void time_slice();

Description	The time_slice function determines if the currently active process has exceeded its current time slice. If the quantum value has been exceeded the dispatcher is called.	
Arguments	None	
Return Value	None	

dispatcher

void dispatcher();

Description	The dispatcher function is the workhorse of the Scheduler. This is where the kernel decides which process to run next and changes context to the next process to run if context needs to change from the currently running process.	
Arguments	None	
Return Value	None	

signaled

int signaled();

Description	Checks if the calling process has been signaled.	
Arguments	None	
Return Value	int	Returns 1 if the process has been signaled, or 0 otherwise..

block

```
int block(int block_status);
```

Description	This function blocks the calling process and sets the processes status in the process table to the value specified in <i>block_status</i> . The value of <i>block_status</i> must be greater than 10. The values of 0-10 are reserved for internal kernel process states.								
Arguments	<i>block_status</i>	The new status value assigned to the blocked process.							
Return Value	int	The function returns 0 upon successful creation of the new process. <table><tr><th>Return Value</th><th>Result</th></tr><tr><td>0</td><td>Success</td></tr><tr><td>-5</td><td>The process was signaled while blocked.</td></tr></table>		Return Value	Result	0	Success	-5	The process was signaled while blocked.
Return Value	Result								
0	Success								
-5	The process was signaled while blocked.								

NOTES:

If this function is called with a value equal to or less than 10, the kernel should be halted with a code of 1:

```
stop(1);
```

Use integer values 0-10 to keep track of the state of processes in the process table.

unblock

```
int unblock(int pid);
```

Description	The unblock function returns a blocked process to the ready to run state.								
Arguments	<i>pid</i>	The ID of the process to unblock.							
Return Value	int	The function returns 0 upon success. Otherwise, an error code is returned according to the following table: <table><tr><th>Return Value</th><th>Result</th></tr><tr><td>0</td><td>Success</td></tr><tr><td>-1</td><td>The process specified by <i>pid</i> is not valid or is not blocked.</td></tr></table>		Return Value	Result	0	Success	-1	The process specified by <i>pid</i> is not valid or is not blocked.
Return Value	Result								
0	Success								
-1	The process specified by <i>pid</i> is not valid or is not blocked.								

get_start_time

int get_start_time();

Description	This function returns the start time of the process in microseconds.	
Arguments	None	
Return Value	int	The function returns the calling process's start time.

cpu_time

int cpu_time();

Description	The cpu_time function returns the amount of time in milliseconds that the current process has spent on the CPU.	
Arguments		
Return Value	int	The function returns the calling process's cpu time.

read_clock

uint32_t read_clock();

Description	This function returns the current value of the system clock using the THREADS system_clock function.	
Arguments	None	
Return Value	uint32_t	The function returns the value of system_clock.

display_process_table

```
void display_process_table(void);
```

Description	This function displays all non-empty processes in the process table. The displayed information includes the process Id (PID), parent PID, Priority, status, number of children, cpu time, and process name.	
Arguments	None	
Return Value	None	

NOTES:

Here is a sample of the output produced by **display_process_table**:

PID	Parent	Priority	Status	# Kids	CUtime	Name
1	-1	0	READY	0	0	watchdog
2	-1	5	WAIT BLOCK	4	0	Scheduler
7	2	3	READY	0	696	SchedulerTest05-Child5
8	2	3	READY	0	712	SchedulerTest05-Child6
9	2	3	READY	0	711	SchedulerTest05-Child7
10	2	3	RUNNING	0	656	SchedulerTest05-Child8

5. Interrupts and Time Slice

The Scheduler must set an interrupt handler for notifications of the timer interrupt. *This is the only interrupt handler used in the Scheduler*, the other interrupts are used in future iterations of the kernel. Therefore, make sure to initialize only the timer interrupt handler and set the remaining handlers to NULL. Set the interrupt handlers using the THREADS `get_interrupt_handlers` function.

See the THREADS User Manual for details on the clock resolution and the frequency of the timer interrupt.

The time-slicing quantum for the kernel is 80 milliseconds.

6. Test Harness

The Scheduler Starter Project is equipped with approximately 30 test processes that each exercise the kernel's Scheduler features and functions. These tests come in the form of a unique version of the `SchedulerEntryPoint` function. Collectively, these test cases represent a comprehensive test harness that validates the proper implementation of the kernel's process management features.

Use the test functions and provided output data to better understand the working mechanisms of the Scheduler. The provided output demonstrates proper operation of the kernel and a properly built scheduler should produce identical, or nearly identical output.

```
SchedulerTest02: started
SchedulerTest02: after spawn of child with pid 3
SchedulerTest02: waiting for child process
SchedulerTest02-Child1: started
SchedulerTest02-Child1: performing spawn of first child
SchedulerTest02-Child1-Child1: started
SchedulerTest02-Child1-Child1: quitting
SchedulerTest02-Child1: spawn of first child returned pid = 4
SchedulerTest02-Child1: performing spawn of second child
SchedulerTest02-Child1-Child2: started
SchedulerTest02-Child1-Child2: quitting
SchedulerTest02-Child1: spawn of second child returned pid = 5
SchedulerTest02-Child1: performing first join.
SchedulerTest02-Child1: exit status for child 4 is -3
SchedulerTest02-Child1: performing second join.
SchedulerTest02-Child1: exit status for child 5 is -3
SchedulerTest02: exit status for child 3 is -3
All processes completed.
```

Figure 2 Reference output for Test02

Figure 2 illustrates the reference output for Test02. Take the time to study the source code of each test and examine the reference output to gain an understanding of the kernel's functionality.

7. Restrictions and Submission

The Scheduler implementation should rely on the C standard libraries and must not use any Windows-specific functionality, including any part of the Win32 API. Avoid the use of the `sleep()` function. When displaying output to the console, use the `console_output()` function rather than `printf()` to maintain proper order of printed text.

Do not modify the provided *THREADSLib.h* file or the provided *ScheduLer.h* file. These files define the interfaces for THREADS and the Scheduler library, respectively. Any data structures or type definitions used in your kernel should be contained in *ScheduLer.c*, *Processes.h*, or in a source file of your choosing.

Submissions are made for the milestone assignment and for the final (completed) scheduler assignment. Submit only the files required to build your Scheduler library. This includes any .h and .c files that make up your kernel. In the Starter Project, these files are named *ScheduLer.c* and *Processes.h*. You may add additional source files to the Scheduler project, just make sure you submit everything needed to build your program. Do not submit any project files or any of the other provided files such as *THREADSLib.h* or *ScheduLer.h*.

Appendix A. Glossary

- **API (Application Programming Interface)** - A set of functions and protocols that allows developers to interact with and utilize the features of the THREADS framework.
- **Bootstrap Function** - The initial function executed by THREADS upon startup. It sets up the operating system Scheduler, initializes core processes, and begins execution.
- **Context Switching** - The process of saving the state of a currently running process and restoring the state of a different process to allow multitasking.
- **Device Control Block** - A structure containing information needed to perform operations on I/O devices, such as commands and data buffers.
- **Interrupt** - A signal to the processor indicating that an event requires immediate attention. THREADS supports timer, I/O, exception, and system call interrupts.
- **Interrupt Vector** - An array of function pointers used to handle interrupts. Each entry corresponds to a specific type of interrupt.
- **I/O (Input/Output)** - The communication between the THREADS kernel and peripheral devices like disks or terminals.
- **Kernel Mode** - A CPU mode with unrestricted access to system resources. Used by the THREADS kernel for operations requiring full system control.
- **Processor Status Register (PSR)** - A simulated register representing the CPU's current state, including flags for interrupts and operating mode (kernel/user).
- **Scheduler** - The component responsible for managing and prioritizing process execution in THREADS.
- **System Clock** - A simulated timer in THREADS that provides the current time in milliseconds since the kernel's initialization.
- **Thread** - A unit of execution within a process. In THREADS, threads are managed through contexts.
- **THREADS** - Acronym for Threaded Heuristic Research, Education, And Development Simulator. A software-based platform for simulating OS functionality.
- **User Mode** - A CPU mode with restricted access to system resources, used for running user-level processes.
- **Watchdog Process** - A core process in THREADS responsible for monitoring system health and ensuring essential operations are functioning correctly.